STI Portal vision and joinup writeup Tristan

Tristan

2026-07-30

Contents

1 Technical requirements **1**
- 1.1 Purpose & scope . . . 1
- 1.2 Glossary . . . 2
- 1.3 Constraints & assumptions . . . 2
  - 1.3.1 **PROPOSED** Drupal engine is retained . . . 2
  - 1.3.2 **PROPOSED** FAO GCP hosting . . . 3
  - 1.3.3 **PROPOSED** No regression of ATIO curation . . . 3
  - 1.3.4 **PROPOSED** Interoperability with FAO infrastructure . . . 3
  - 1.3.5 **PROPOSED** Deliver the promised scope . . . 3
- 1.4 Functional requirements . . . 3
  - 1.4.1 Data agglomeration AGG . . . 3
  - 1.4.2 Curation CUR . . . 3
  - 1.4.3 Access & interoperability API . . . 4
  - 1.4.4 Networking NET . . . 5
  - 1.4.5 Indicators IND . . . 6
- 1.5 Non-functional / quality requirements . . . 6
  - 1.5.1 **PROPOSED** Interoperability & FAIR . . . 6
  - 1.5.2 **PROPOSED** Provenance & AI-origin auditability . . . 6
  - 1.5.3 **PROPOSED** High-scale ingestion performance . . . 6
  - 1.5.4 **PROTOTYPE** Collection-level access control . . . 6
  - 1.5.5 **PROPOSED** Maintainability . . . 6

2 MVP and working implementation **7**
- 2.1 Problem statement . . . 7
- 2.2 Solutions/ improvement . . . 9

1 Technical requirements

1.1 Purpose & scope

This register states **what** the redesigned STI Portal / ATIO KB must do and which qualities it must have. It specifically discusses how joinup (an EC-curated drupal distribution) could address the stated project aims. It deliberately excludes:

- business rationale and funding logic → CAPEX proposal,
- architecture and technology choices (joinup, CQRS dual-DB, triplestore, Solr, Milvus, the JSON import schema) → Design Doc v4.2 and Architecture Decision Records (ADRs).

Each requirement referencing a design choice points to the ADR that owns it, so requirements stay stable while the "how" evolves under agile.

1.2 Glossary

AFSI Agrifood System Innovation — the **content** / knowledge unit. This encompasses both original records and ATIO innovation records or other representations in the platform.

AFSA Agrifood System Actor — a **person or organisation** (policymaker, innovator, farmer rep, researcher, funder).

Claim an AAA-style statement a collection makes about an AFSI; stored alongside slim core fields rather than forced into a fixed schema.

Collection a data source's own body of records, retaining its own vocabulary/taxonomy.

Connected a notebook-style entry of a AFSA in another AFSA's profile

Relevant (to an actor) Topic or interest matches AFSA's interests

AAA "Anyone can say Anything about Anything" — the semantic-web principle underlying the claims model.

FAIR Findable, Accessible, Interoperable, Reusable.

SKOS Simple Knowledge Organization System (vocabulary/taxonomy exchange).

RAG Retrieval-Augmented Generation — an application pattern over a vector-search endpoint, not itself a query interface.

ADR Architecture Decision Record.

PROPOSED not yet accepted by the team, not yet built

AGREED accepted by the team

PROTOTYPE built in my current working mvp (joinup-based), not yet accepted, neither in production

BUILT accepted and built in production

DEFERRED don't do / maybe later

ATIO(t) ATIO team's responsibility

1.3 Constraints & assumptions

These are the *givens* I was working based off on. Changing them would change the outlook on the project and its aims significantly.

1.3.1 PROPOSED Drupal engine is retained

The Drupal engine remains the content-handling and orchestration layer (team skills, consultant pool, de-facto public-sector standard).

1.3.2 PROPOSED FAO GCP hosting

The system is hosted on FAO Google Cloud Platform, with edge → review → production CD.

1.3.3 PROPOSED No regression of ATIO curation

The redesign shall not remove any curation capability the ATIO team currently relies on, and existing ATIO curation workflows (analysis, synthesis, standardisation) shall remain operational. (This absorbs the old "existing workflows keep working" acceptance line, which was a non-regression check rather than a forward requirement.)

1.3.4 PROPOSED Interoperability with FAO infrastructure

The system must interoperate with existing FAO infrastructure and vocabularies (AGROVOC, FAOSTAT, Agritech Observatory, ...)

1.3.5 PROPOSED Deliver the promised scope

The redesign must deliver what the CAPEX proposal and ATIO Design Doc outline. It should at this point not go beyond those two, but first deliver and present the MVP satisfying the demands of the two documents.

1.4 Functional requirements

1.4.1 Data agglomeration AGG

1. **PROTOTYPE** Import preserves original key–value pairs The drupal migrate system shall import a source record supplied in a supported format, preserving the source's original field names and values without mapping them onto a fixed schema.
   - Acceptance: five structurally different sources load through the same path, and every original key–value pair is retrievable afterwards.
   - Supported formats are structured data in JSON.
2. **PROTOTYPE** Data is parsed into standard JSON schema A standard json schema is developed. An interactive CLI tool shall read in tabular (xlsx, csv) data and let the user interactively set which columns are categorical, which are multi-valued and what separators they use. It then emits the standard JSON digest of that data.
   - Acceptance: five structurally different sources can be parsed through the tool.
   - Alternative: More complete agentic workflow that let's user interactively analyze and digest the incoming data structure.

1.4.2 Curation CUR

1. **PROTOTYPE** Non-developer curator can author records via GUI A content curator without software-development skills shall be able to create, edit, and submit an innovation record for review through a graphical interface, without writing code or running CLI commands.
   - Acceptance: a non-developer completes record creation → submission in a usability test with no CLI use.
2. **PROTOTYPE** Maintainer can curate SKOS relationships A data curator can match semantic dimensions across collections using SKOS (e.g. 'Technological Readiness' of collection A similar to 'Readiness' of collection B).

- Acceptance: a non-developer loads two collection's properties, matches them to one another using skos terms and exports a working SPARQL query able to import those matches into the triplestore.

1.4.3 Access & interoperability **API**

1. **PROTOTYPE** SPARQL endpoint per collection Each data collection shall be queryable by external machine clients via a SPARQL endpoint, returning records annotated with that collection's own vocabulary.
   - Acceptance: These queries pass and return the same information stored in SQL

```
### Get collections
  PREFIX dct:  <http://purl.org/dc/terms/>
PREFIX dcat: <http://www.w3.org/ns/dcat#>

SELECT ?collection ?title WHERE {
  GRAPH <http://data> {
    ?collection a dcat:Catalog .
    OPTIONAL { ?collection dct:title ?title }
  }
} ORDER BY ?title
### Solutions per collection
  PREFIX dct:  <http://purl.org/dc/terms/>
PREFIX dcat: <http://www.w3.org/ns/dcat#>
PREFIX sti:  <https://sti-portal.fao.org/ontology#>

SELECT ?collection ?title
       (COUNT(DISTINCT ?fwd) AS ?viaDcatDataset)
       (COUNT(DISTINCT ?rev) AS ?viaInCollection)
WHERE {
  GRAPH <http://data> {
    ?collection a dcat:Catalog .
    OPTIONAL { ?collection dct:title ?title }
    OPTIONAL { ?collection dcat:dataset ?fwd . ?fwd a sti:Innovation }
    OPTIONAL { ?rev a sti:Innovation ; sti:inCollection ?collection }
  }
} GROUP BY ?collection ?title ORDER BY DESC(?viaInCollection)
### Claims per collection
PREFIX sti: <https://sti-portal.fao.org/ontology#>

SELECT (COUNT(?claim) AS ?totalClaims)
       (COUNT(DISTINCT CONCAT(STR(?property), "|", STR(?value))) AS
       ↪  ?uniqueClaims)
       (COUNT(DISTINCT ?property) AS ?properties)
       (COUNT(DISTINCT ?solution) AS ?solutionsTouched)
WHERE {
  GRAPH <http://data> {
    ?claim sti:claimedBy <COLLECTION-URI> ;
           sti:claimProperty ?property .
    OPTIONAL { ?claim sti:claimRefValue     ?ref }
    OPTIONAL { ?claim sti:claimLiteralValue ?lit }
```

```
    BIND(COALESCE(?ref, ?lit) AS ?value)
    OPTIONAL { ?solution sti:hasFieldClaim ?claim }
  }
}
```

---

2. **PROTOTYPE** REST API per collection Each data collection shall be queryable by external machine clients via a REST API, returning records annotated with that collection's own vocabulary.
   - Acceptance: REST API returns json using curl. REST API is successfully queried by another drupal instance (e.g. custom block)
3. **PROPOSED** Vector / semantic-search endpoint per collection The platform shall expose a vector/semantic-search endpoint over each collection, suitable for consumption by RAG clients.
   - Acceptance: A rag agent pointed at the VecDB returns relevant innovations when prompted.
4. **PROTOTYPE** Collection-level landing page The platform shall present one landing page per collection listing this collections AFSIs and filtered search based on that collection's taxonomy.
   - Acceptance: Landing pages exist, load and filter fast (<3s) and display the relevant (categorical) claims for that datasource
5. **PROTOTYPE** RDF Data Model The platform shall employ a data model that matches both Unit 1 and Unit 5 requirements.
   - Acceptance: ttl files provided by Valeria and Tristan matched and represented in the triplestore

1.4.4 Networking NET

Networking depends on solid implementation of the core STI portal (AGG, CUR, API). Currently open social is used to achieve this; discussion on other solutions would be worthwhile.

1. **PROPOSED** Actors linkable to innovations and to one another Actors (AFSA) should be linkable to the innovations (AFSI) held in central storage, and to one another.
2. **PROPOSED** Actor-to-actor messaging An actor shall be able to send a message to another actor through the platform.
3. **PROTOTYPE** Groups / communities of practice Actors shall be able to form and join groups / communities of practice on the platform.
4. **PROPOSED** Surface relevant innovations to an actor An actor should be surfaced innovations relevant to them.
5. **PROPOSED** Surface relevant actors to an actor An actor should be surfaced other actors relevant to them.
6. **PROTOTYPE** Challenge organization An actor (AFSA) with an organiser role should be able to create an innovation challenge on the platform and collect idea submissions from other actors.

- Out of scope: assessment, scoring, or ranking of submissions (collection only, no adjudication).
- Acceptance: an organiser creates a challenge; a second actor submits an idea to it; the submission is stored and visible to the organiser; no scoring/ranking UI is present.

1.4.5 Indicators IND

1. **PROPOSED** Display external indicators about an innovation The platform shall display ASTI indicators about a country drawn from external sources (e.g. FAOSTAT).
   - Acceptance: A custom block successfully queries the FAOSTAT API rendering the correct (i.e. matches URL/search) country's ASTI indicators as separate graphs.
2. **PROPOSED** User-defined assessment criteria Data owners and platform users may define their own assessment criteria for innovations.
   - Acceptance: The nudge unit has developed a series of indicators. If those can be added in the data model, rendered and edited by people with the correct drupal roles, the data model and implementation work.

1.5 Non-functional / quality requirements

1.5.1 PROPOSED Interoperability & FAIR

The data should be FAIR; specifically FAO colleagues should be able to use the data by filtered search or API calls.

- Acceptance: The data model and storage is shared with at least one other FAO project, in coordination with CSI.
- Example: Agritech Observatory

1.5.2 PROPOSED Provenance & AI-origin auditability

Claims can be versioned, the AI flags and assessment are successfully ported from the STI portal.

1.5.3 PROPOSED High-scale ingestion performance

Loading a collection of up to 2000 innovations should take not more than 6 h along the whole pipeline from receiving the non-standard but structured data, to parsing into standard JSON and import into SQL/RDF/VecDB.

1.5.4 PROTOTYPE Collection-level access control

Every collection's claims should be editable by profiles that have the role to curate this collection.

- Note: Joinup support this already

1.5.5 PROPOSED Maintainability

The joinup-based platform should be maintainable by a 0.5 FTE Drupal/PHP developer (security updates, patches, maintenance of custom modules following updates).

2 MVP and working implementation

I did a hands-on technical exploration and implementation of the platform described to my understanding in the CAPEX project and the ATIO design doc. In close coordination with Valeria (on knowledge graphs, data modelling and ATIO workflow needs) and Andrea (Drupal best practices, load testing) I built an MVP that satisfies the technical requirements stated above. This was a two-way process: building, discovering the requirements, documenting the requirements, updating the build. I started with market research on what platforms and libraries come close to achieving our desired goals and identified joinup as a good solution.

It can serve as the heart of our STI infrastructure and handle all things data storage, content curation and -display and data access.

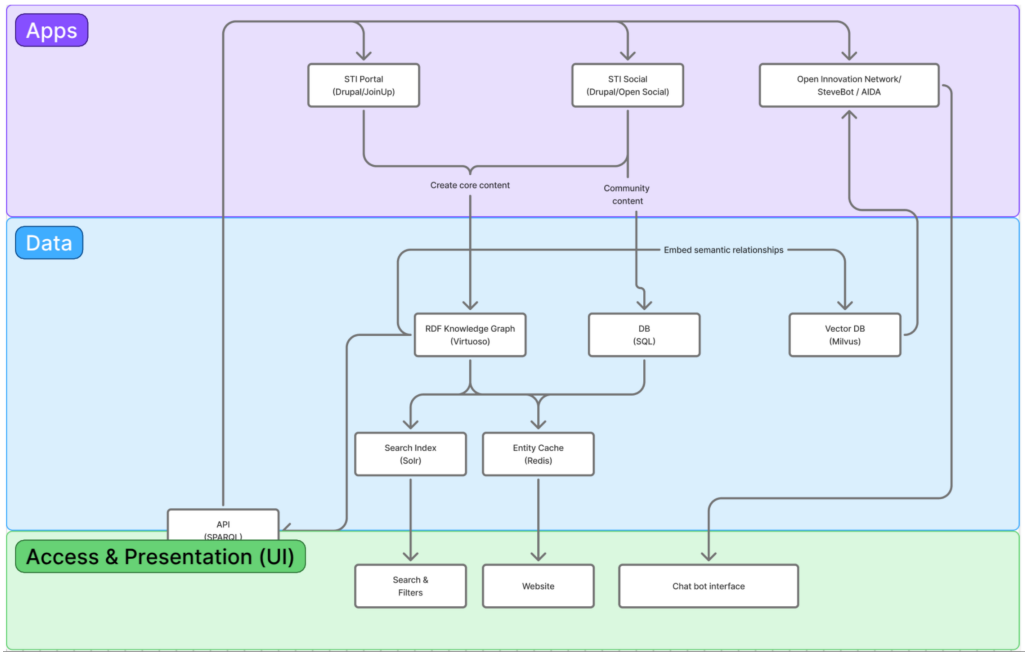


Figure 1: Proposed architecture of a drupal-based knowledge graph platform.

2.1 Problem statement

Currently, the STI portal is far from delivering what is promised in the capex and ATIO design docs. Central to this problem is the high complexity of the data model that makes loading new content in unnecessarily cumbersome:

There are many points of optimization, but a central one that can solve almost all issues at one is to adopt a semantic web data model. Anyone can say anything about anything (AAA). This is currently the de-facto state of things when it comes to records about innovations within and without FAO. Forcing all these different frameworks into ATIO's 80-field $\text{source}_{\text{record}}$/innovation data type makes everything very complicated from the start. I suggest to separate these concerns: Data uptake must be easy and fully record the data source's meta information. Then, using SKOS and ATIO's curation approach using LLMs these different ways to talk about innovations can be brought together in the ATIO knowledge base. AI-based curation should ideally be done outside of Drupal & the current ECA framework is too brittle to support high-scale data import and curation. While we can keep it, I propose an alternative more scalable pathway for data loading that leverages drupal's migrate. AI enrichment at whichever stage should follow a self-controlling algorithm.

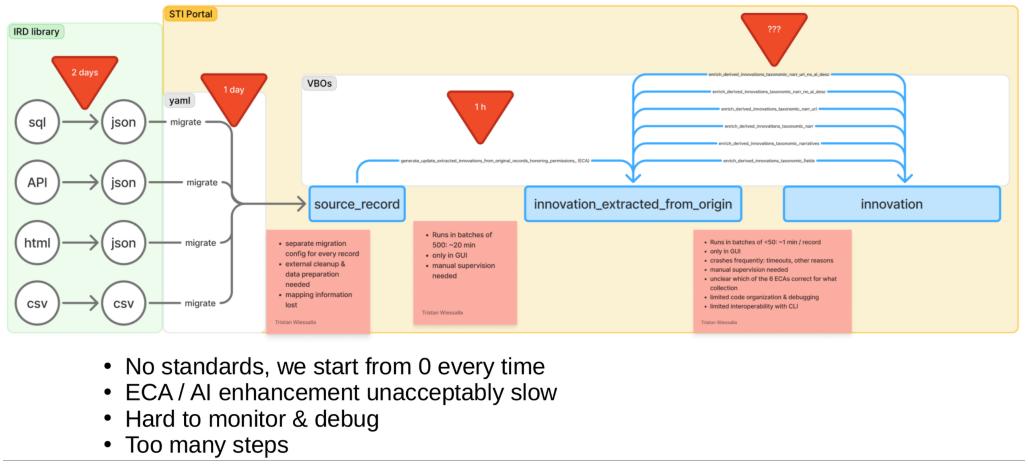


Figure 2: Current data loading pipeline from source to innovation displayed on web page

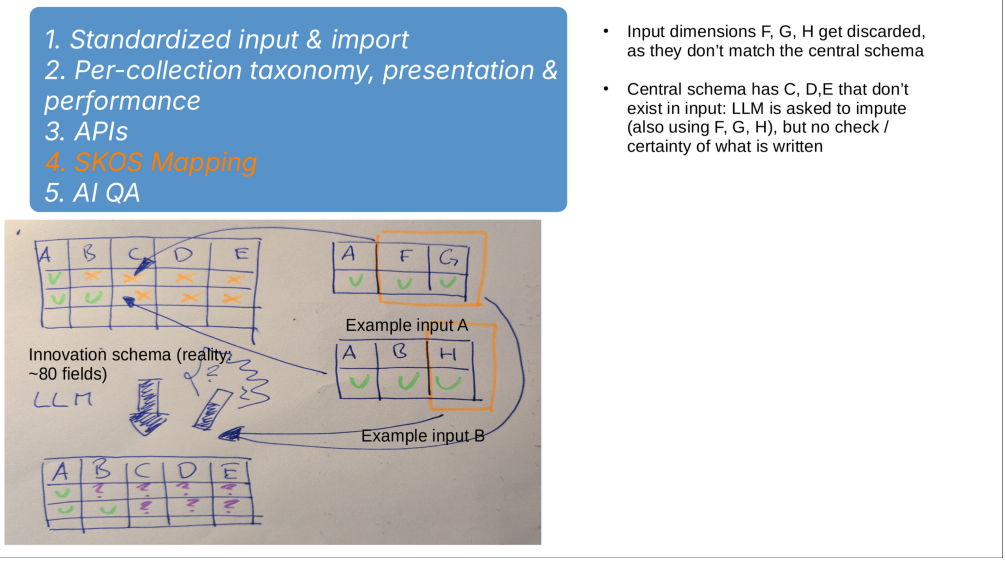


Figure 3: Abstract semantic data mapping representation as implemented currently

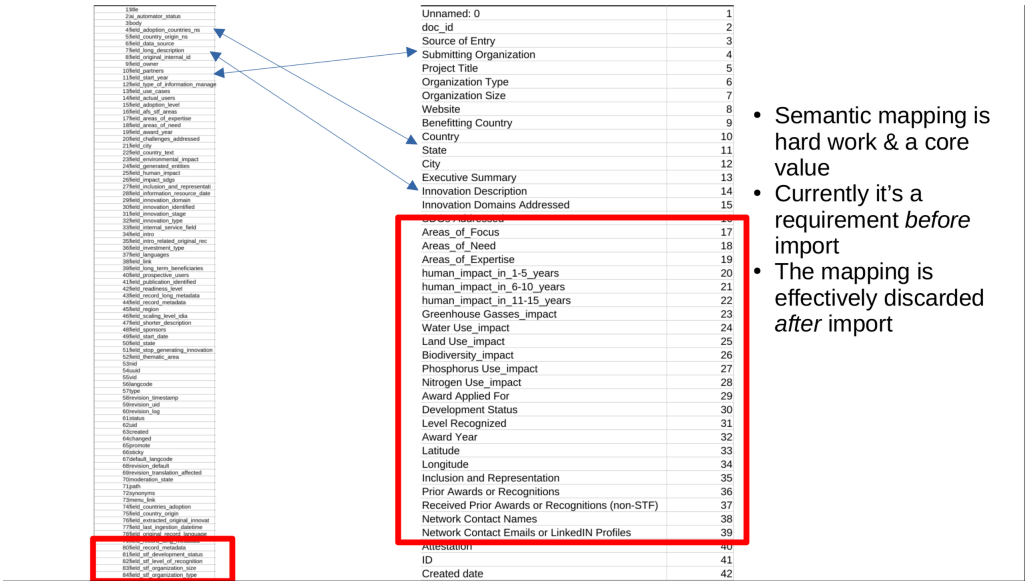


Figure 4: Data mapping hands-on example

2.2 Solutions/ improvement

1. Standardized json schema for data import (avoids steps 1 & 2 in diagram above)
   - Core data (required) clearly separated from meta-data (claims)
   - No custom migrations needed anymore drupal-side
   - Data curation can be done using python/R/LLMs
2. Adopt joinup's CQRSknowledge graph architecture (Delivers the expectation to federate with AGROVOC etc and provide reusable taxonomies)
3. AAA principle: Reflect what each data collection says about their innovations (solves the brittleness and high linearity of steps 3 ff of the problem statement figure)
   - Keep their dimensions
   - Provide a collection landing page re-using these dimensions as filters
   - Import data as-is, curate later (especially ATIO)
4. Have any AI-enrichment be automatically double-checked (more accountability, origin tracing and quality for automatically generated content)
   - Creating content without any safeguards / assessment is extremely risky and lead to quite some problems
   - Implemented algorithm (for now)
5. Allow connecting to ollama etc (cheaper, more control on long-term training & fine-tuning)
6. Interface with other apps through apis (event based)

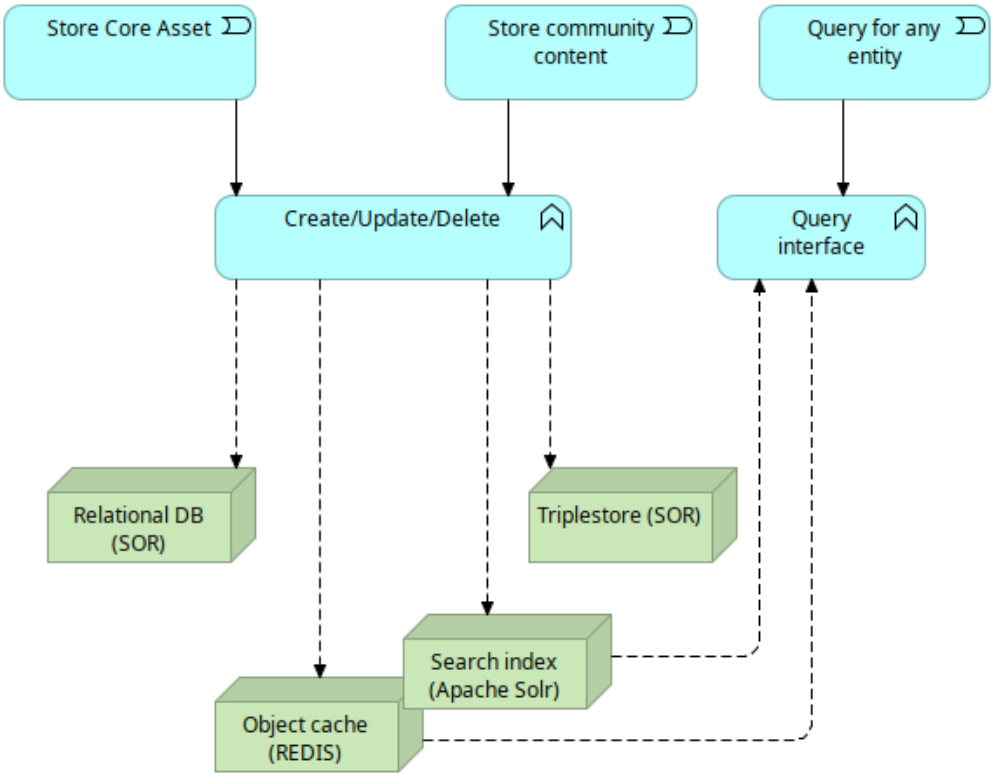


Figure 5: Joinup data architecture

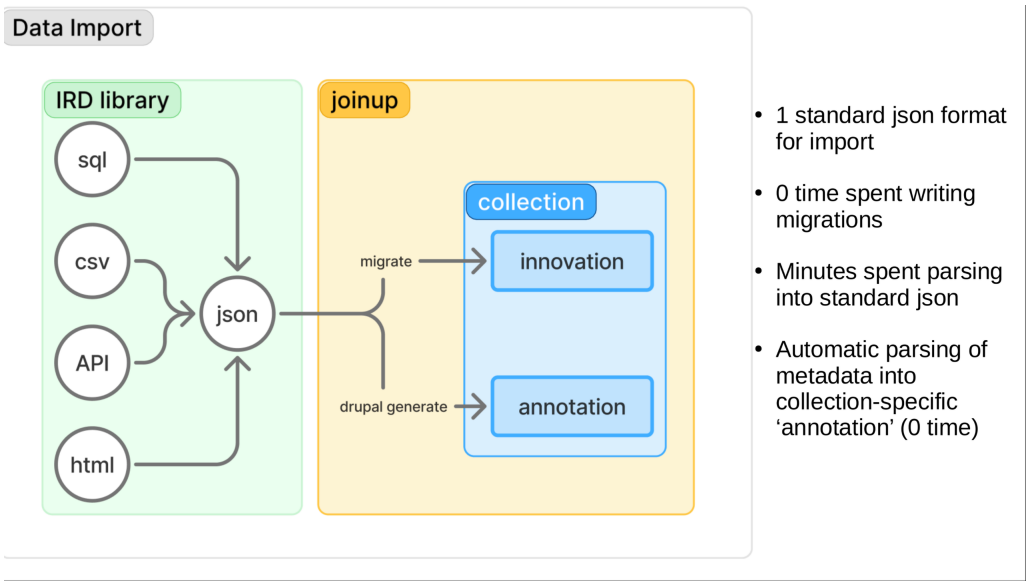


Figure 6: Standard json format and downstream logic.

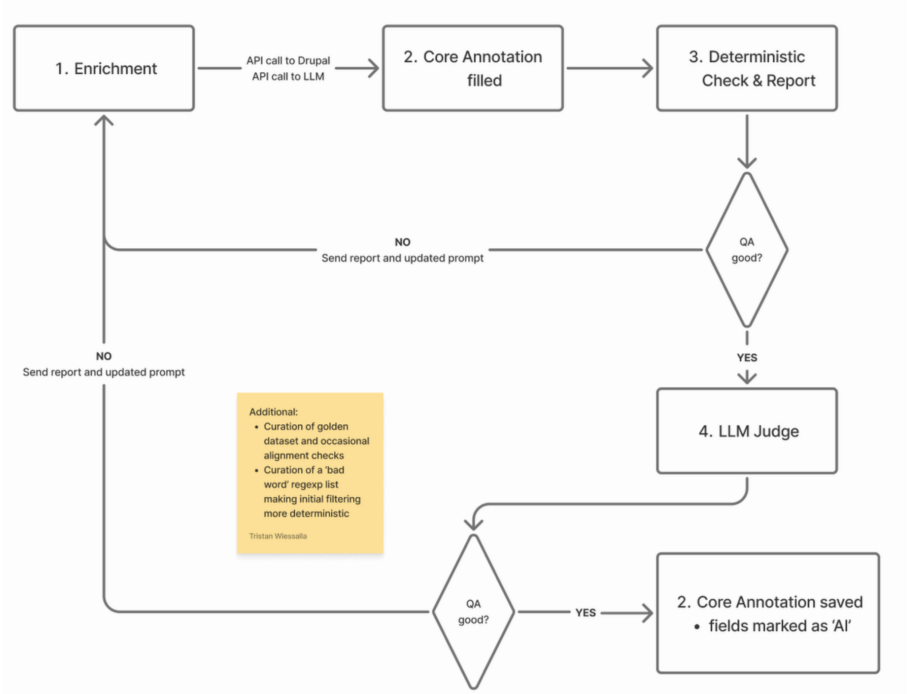


Figure 7: QA algorithm for AI-enrichment (improvement) of content.

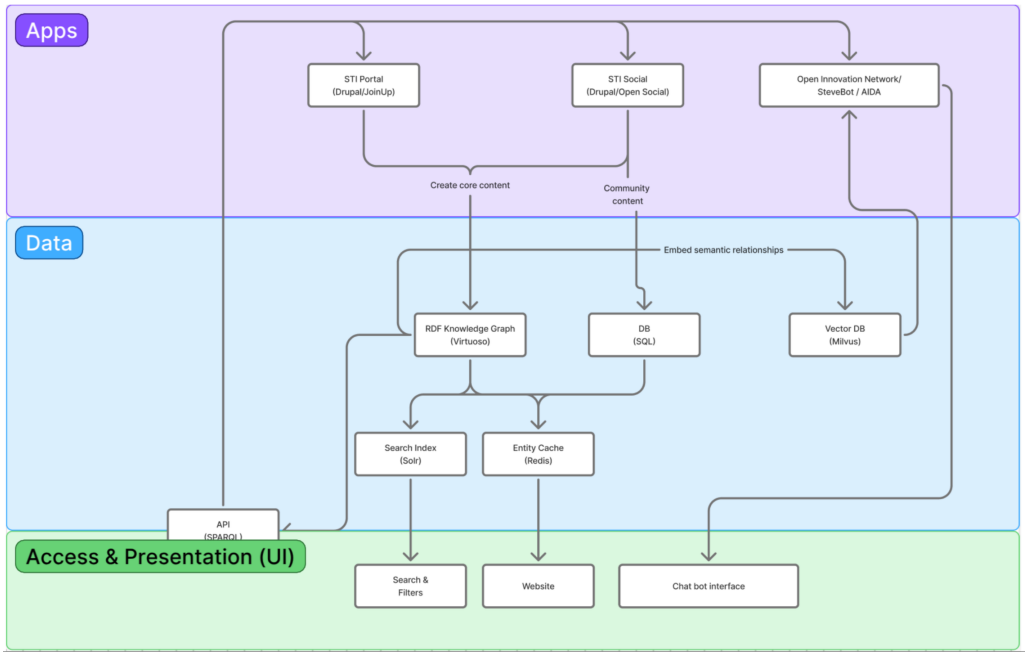


Figure 8: Proposed architecture of a drupal-based knowledge graph platform.